

Fundamentos de Bioinformática e Linux

Dia 1 do minicurso Bioinformática Básica e Aplicada à Genômica e Transcriptômica

Profa Dra Helena R. S. D’Espindula

Contents

1	Introdução Teórica	3
1.1	Apresentação	3
1.2	O que é Bioinformática?	3
1.3	Por que usar Linux em Bioinformática?	3
1.4	Sequenciamento de Nova Geração (NGS)	3
1.5	Fluxo de trabalho em bioinformática:	3
1.6	Terminal vs. Interface gráfica:	3
1.7	O Que é um Pipeline?	4
1.8	Construindo um Pipeline: Ferramentas e Scripts	4
1.9	Linguagens de Programação na Bioinformática	4
1.10	Recomendações para Reprodutibilidade	5
1.11	Programas e Ferramentas de Interesse em Bioinformática	6
1.12	Programas e Ferramentas de Interesse em Bioinformática - Enfoque em procariotos	6
1.13	Programas e Ferramentas de Interesse em Bioinformática - Enfoque em vertebrados/humanos	7
2	Introdução ao Linux e Linha de Comando (Atividades Práticas)	8
2.1	Acessando o Linux do Windows	8
2.2	Navegação Básica - Comando <code>pwd</code>	13
2.3	Navegação Básica - Comando <code>cd</code>	13
2.4	Manipulação de Diretórios e Arquivos - Comando <code>ls</code>	13
2.5	Manipulação de Diretórios e Arquivos - Comando <code>mkdir</code>	13
2.6	Manipulação de Diretórios e Arquivos - Comando <code>rmdir</code>	14
2.7	Abrindo um programa pela linha de comando - Comando <code>evince</code>	14
2.8	Navegação Básica - Comando <code>man</code>	14
2.9	Navegação Básica - Comando <code>ssh</code>	15
2.10	Manipulação de Diretórios e Arquivos - Comando <code>touch</code>	15
2.11	Manipulação de Diretórios e Arquivos - Comando <code>echo</code>	15
2.12	Manipulação de Diretórios e Arquivos - Comando <code>cat</code>	15
2.13	Manipulação de Diretórios e Arquivos - Comando <code>cp</code>	16
2.14	Manipulação de Diretórios e Arquivos - Comando <code>mv</code>	16
2.15	Manipulação de Diretórios e Arquivos - Comando <code>rm</code>	16
2.16	Download de Arquivos - Comando <code>wget</code>	17
2.17	Manipulação de Diretórios e Arquivos - Comando <code>head</code> e <code>tail</code>	17
2.18	Limpar a tela - <code>clear</code> ou <code>Ctrl+L</code>	18
2.19	Manipulação de Diretórios e Arquivos - Comando <code>less</code>	18
2.20	Transferência de Arquivos entre computadores - Comando <code>scp</code>	18
2.21	Monitorando processos - Comando <code>top</code>	18
2.22	Criação de Ambiente de Trabalho – Comandos <code>screen</code> ou <code>tmux</code>	20
2.23	Compactação de Arquivos - Comando <code>gzip</code> e <code>gunzip</code>	21

2.24	Compactação de Arquivos - Comando tar	21
2.25	Monitoramento de tempo de execução - Comando time	21
2.26	Processamento de Dados em Tabelas - Comandos cut , sort , uniq , wc , grep	22
2.27	Permissões e Ambiente - Comando chmod	22
2.28	Permissões e Ambiente - Comando which	23
2.29	Permissões e Ambiente - Comando history	23
2.30	Encontrar Arquivos e/ou Diretórios - Comando find	23
2.31	Outros Exemplos úteis de comandos em Bioinformática	24
2.32	Atividade extra	25

1 Introdução Teórica

1.1 Apresentação

- Apresentação dos ministrantes do curso
- Apresentação dos alunos

1.2 O que é Bioinformática?

- Integração de biologia, informática e estatística para análise de dados biológicos.
- Aplicações: genômica, transcriptômica, filogenia, metagenômica, etc.

Glossário de Termos de Bioinformática: https://staphb.org/resources/2019-12-18-binfo_glossary.html

1.3 Por que usar Linux em Bioinformática?

- Maior compatibilidade com ferramentas científicas.
- Mais controle sobre o ambiente de execução.
- Reprodutibilidade.

1.4 Sequenciamento de Nova Geração (NGS)

O NGS refere-se a diversas abordagens de alto rendimento para o sequenciamento de DNA, baseadas no conceito de processamento massivo em paralelo.

Também é conhecido como:

- Sequenciamento em paralelo massivo
- Sequenciamento de segunda geração
- Sequenciamento de alto rendimento

1.5 Fluxo de trabalho em bioinformática:

1. Coleta de dados (podem ser Experimentos ou *Wet lab*)
2. Pré-processamento (qualidade)
3. Mapeamento e montagem
4. Análise funcional e estatística
5. Interpretação biológica

1.6 Terminal vs. Interface gráfica:

- CLI (Interface de Linha de Comando) ou “Terminal”: rápido, leve, scriptável.
- GUI (Interface Gráfica): amigável, mas menos flexível.

CLI exige mais conhecimento técnico.

- Pode envolver:
 - compilação de código-fonte (C, C++) ou
 - uso de scripts (Python, Perl, Java, R).
- Softwares podem ser instalados manualmente ou via contêineres (ex.: Docker).
- GitHub é um repositório comum para programas de bioinformática.

1.7 O Que é um Pipeline?

Cadeia de ferramentas que conecta etapas analíticas: Saída de uma = Entrada da próxima

Termos relacionados: workflow, fluxo de trabalho

Exemplo de Pipeline Bioinformático Genérico:

1. Controle de qualidade das leituras
2. Determinação de cepa de referência
3. Mapeamento das leituras
4. Detecção de SNPs e indels
5. Montagem de novo
6. Anotação do genoma
7. Construção e análise de árvore filogenética

1.8 Construindo um Pipeline: Ferramentas e Scripts

Linguagens: Bash, Python, Perl

Ferramentas de workflow:

- Snakemake
- Nextflow
- Galaxy
- Conjuntos como o StaPH-B Toolkit

1.9 Linguagens de Programação na Bioinformática

As linguagens de programação são críticas na bioinformática moderna, pois permite automatizar análises, integrar ferramentas e interpretar grandes volumes de dados.

1.9.1 Bash (ou Shell scripting)

- Descrição: Linguagem nativa do terminal Linux.
- Usos: Automação de tarefas, encadeamento de comandos, manipulação de arquivos, execução de pipelines.

Exemplo de uso:

```
for file in *.fastq; do fastqc $file; done
```

- Softwares e ambientes mais usados:
 - Terminal Linux (LXTerminal, GNOME Terminal, Konsole)
 - Shell script em `.sh` via editores simples (nano, vim) ou IDEs como VS Code
 - **Servidores remotos**, clusters e HPC (High Performance Computing)

1.9.2 Python

- Descrição: Linguagem versátil, legível, com grande número de bibliotecas específicas para bioinformática.
- Usos: Análises estatísticas, manipulação de sequências, machine learning, visualização de dados.
- Principais bibliotecas: **Biopython**, **pandas**, **scikit-learn**, **matplotlib**, **seaborn**.

Exemplo de uso:

```
from Bio import SeqIO
for record in SeqIO.parse("genoma.fasta", "fasta"):
    print(record.id, len(record.seq))
```

- Softwares e ambientes mais usados:
 - Jupyter Notebook (local ou via Anaconda)
 - Google Colab
 - VS Code ou PyCharm
 - Terminal com interpretador Python puro

1.9.3 R

- Descrição: Linguagem voltada para estatística e ciência de dados, com amplo uso em análises genômicas e transcriptômicas.
- Usos: RNA-Seq, análises diferenciais, clustering, PCA, visualizações avançadas.
- Principais pacotes: DESeq2, edgeR, ggplot2, tidyverse, limma, biomaRt.

Exemplo de uso:

```
counts <- read.table("counts.txt", header=TRUE, row.names=1)
summary(counts)
```

- Softwares e ambientes mais usados:
 - RStudio Desktop ou Server
 - R via Jupyter Notebooks
 - Bioconductor
 - Terminal com R puro

1.9.4 Outras linguagens e ferramentas comuns

Linguagem / Ferramenta	Usos principais	Ambiente comum
Perl	Scripts legados, regex complexos	Terminal, scripts .pl
C / C++	Desenvolvimento de ferramentas de alto desempenho	Terminal, Makefiles, compiladores
Nextflow / Snakemake	Automação e orquestração de pipelines	Terminal, VS Code, ambientes de workflow
SQL	Consultas a bancos de dados genômicos	MySQL, SQLite, PostgreSQL + GUI

1.10 Recomendações para Reprodutibilidade

1. Documente como cada resultado foi gerado.
2. Evite manipulações manuais de dados.
3. Armazene versões exatas dos softwares usados.
4. Controle de versão de scripts.
5. Registre resultados intermediários.
6. Defina sementes para análises aleatórias.
7. Armazene os dados brutos utilizados nos gráficos.
8. Permita inspeção em camadas dos resultados.

9. Relacione afirmações com dados reais.
 10. Disponibilize publicamente os scripts, execuções e resultados.
- Saiba mais: CDC NGS Quality Initiative

1.11 Programas e Ferramentas de Interesse em Bioinformática

A bioinformática conta com um ecossistema robusto de ferramentas desenvolvidas para diferentes etapas do fluxo de análise de dados biológicos. Abaixo estão alguns dos softwares e plataformas mais utilizados na prática profissional e acadêmica.

1.11.1 Organizações e Plataformas de Apoio

- **StaPH-B**: Grupo estadual de profissionais de bioinformática em saúde pública dos EUA, voltado para capacitação e respostas epidemiológicas.
- **ABiL** (Applied Bioinformatics Laboratory): Parceria entre Georgia Tech e IHRC, Inc., oferece treinamentos e recursos abertos em bioinformática.
- **Apache Guacamole**: Ferramenta web para acesso remoto a servidores bioinformáticos via navegador (usada, por exemplo, pelo CDC).
- **Galaxy**: Plataforma web colaborativa para análises bioinformáticas sem necessidade de programação. Muito usada em contextos educacionais e institucionais (ex: GenomeTrakr).
- **BaseSpace**: Ambiente online da Illumina para gerenciamento e análise de dados gerados em sequenciadores da própria empresa.
- **Nano, VIM**: editores de texto (tipo “bloco de notas”)

1.11.2 Controle de Qualidade e Pré-processamento

- **FastQC**: Avaliação da qualidade de leituras de sequenciamento (ex: FASTQ).
- **Trimmomatic**: Corte de adaptadores e regiões de baixa qualidade em leituras.
- **Mash**: Estima rapidamente a similaridade entre genomas ou metagenomas com base em sketching de k-mers.
- **seqtk**: Ferramenta leve e rápida para processamento dos formatos FASTA ou FASTQ

1.12 Programas e Ferramentas de Interesse em Bioinformática - Enfoque em procariotos

1.12.1 Montagem e Anotação de Genomas

- **SPAdes**: Montagem de novo de genomas a partir de leituras curtas.
- **Shovill**: Wrapper otimizado para SPAdes e outros montadores como Velvet.
- **Unicycler**: Montagem híbrida (curtas + longas) com foco em precisão estrutural.
- **Velvet**: Um dos primeiros montadores para leituras curtas; ainda usado em pipelines legados.
- **MIRA / Lyve-SET**: Ferramentas voltadas para análises de surto e montagem de precisão.
- **PROKKA**: Anotação automática e rápida de genomas bacterianos.
- **Quast**: Avaliação da qualidade de montagens genômicas com estatísticas e comparações.

1.12.2 Análises Filogenéticas e Taxonômicas

- **BLAST**: Busca por similaridade entre sequências em bancos de dados do NCBI.
- **Kraken**: Classificação taxonômica baseada em k-mers, ideal para metagenômica.
- **IqTree**: Inferência filogenética por máxima verossimilhança com suporte a bootstraps rápidos.
- **Lyve-SET**: Inclui análises filogenéticas baseadas em SNPs de alta qualidade (hqSNP).
- **MicrobeTrace**: Ferramenta do CDC para visualização de clusters genéticos em redes de transmissão.

1.12.3 Sorotipagem e Análises Específicas

- **SeqSero**: Identificação de sorotipos de *Salmonella* com base em leituras ou montagens.
- **SISTR**: Alternativa ao SeqSero para tipagem in silico de *Salmonella*.
- **SerotypeFinder**: Sorotipagem de *Escherichia coli* a partir de sequências genômicas.
- **Abricate**: Detecção de genes de resistência e virulência em contigs montados.

1.12.4 Ambientes Gráficos e Comerciais

- **CLC Genomics Workbench**: Ambiente comercial completo para montagem, RNA-seq, metagenômica e visualização, compatível com múltiplas plataformas de sequenciamento.
- **Illumina**: Empresa líder em sequenciamento de nova geração (NGS), fornece não só os equipamentos como também pipelines proprietários e ambientes (ex: BaseSpace).

1.13 Programas e Ferramentas de Interesse em Bioinformática - Enfoque em vertebrados/humanos

1.13.1 Bases de dados

- **Ensembl**: genome browser de genomas de vertebrados. Genômica comparativa, variação de sequências e anotações funcionais.
- **UCSC Genome Browser**: Visualização de dados genômicos, ferramentas de manipulação de sequências e download de dados.
- **NCBI Map Viewer**: ferramentas para explorar dados do genoma humano e dados correlatos.

1.13.2 Mapeadores de genoma (NGS)

- **BWA** (Burrows-Wheeler Aligner e derivados): mapeador de sequências curtas de NGS em um genoma de referência.
- **Bowtie** e **Bowtie2**: Mapeador de sequências curtas de NGS em genoma de referência.
- **hla-mapper**: mapeamento otimizado de genes HLA.
- **minimap2**: mapeamento de sequências longas (PacBio, Nanopore) e curtas.

1.13.3 Anotadores

- **ANNOVAR**: Anotação funcional e estrutural de variantes genéticas.
- **VEP**: Anotação funcional e estrutural de variantes genéticas.
- **SnpEff**: Anotação funcional e estrutural de variantes genéticas.

1.13.4 Utilitários para manipular e analisar arquivos intermediários (BAM/SAM e VCF)

- **bcftools**: conjunto de ferramentas para visualizar, filtrar, analisar e processar dados de sequências mapeadas, bem como variantes chamadas.
- **SAMtools**: conjunto de ferramentas para visualizar, filtrar, analisar e processar dados de sequências mapeadas.
- **BEDtools**: conjunto de ferramentas para manipulação de intersecção, união, subtração e outras operações em arquivos de regiões genômicas (BED, GFF, VCF).

1.13.5 Ferramentas de visualização de sequências

- **IGV** (Integrative Genômica Viewer): visualizador integrativo de dados genômicos para analisar mapeamentos de reads, variantes, genes e outras anotações genômicas de contexto visual.

2 Introdução ao Linux e Linha de Comando (Atividades Práticas)

2.1 Acessando o Linux do Windows

Opções:

- Dual-boot
- Pen-drive boot
- Máquina Virtual / Subsistema
- Acesso de Linux remoto (Servidor):
 - Via interface gráfica
 - Via linha de comando do windows cmd

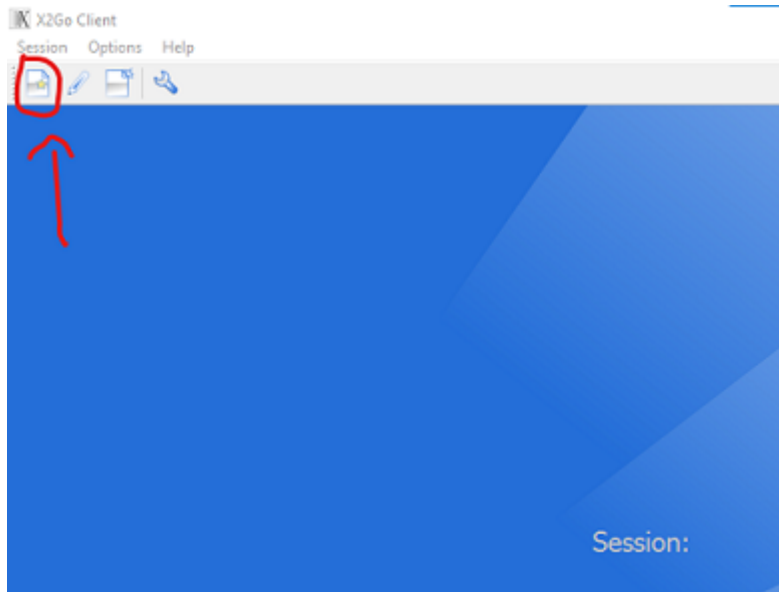
Para esse curso vamos fazer acesso remoto com interface gráfica no servidor de Bioinformática do ICC.

Dados do Servidor:

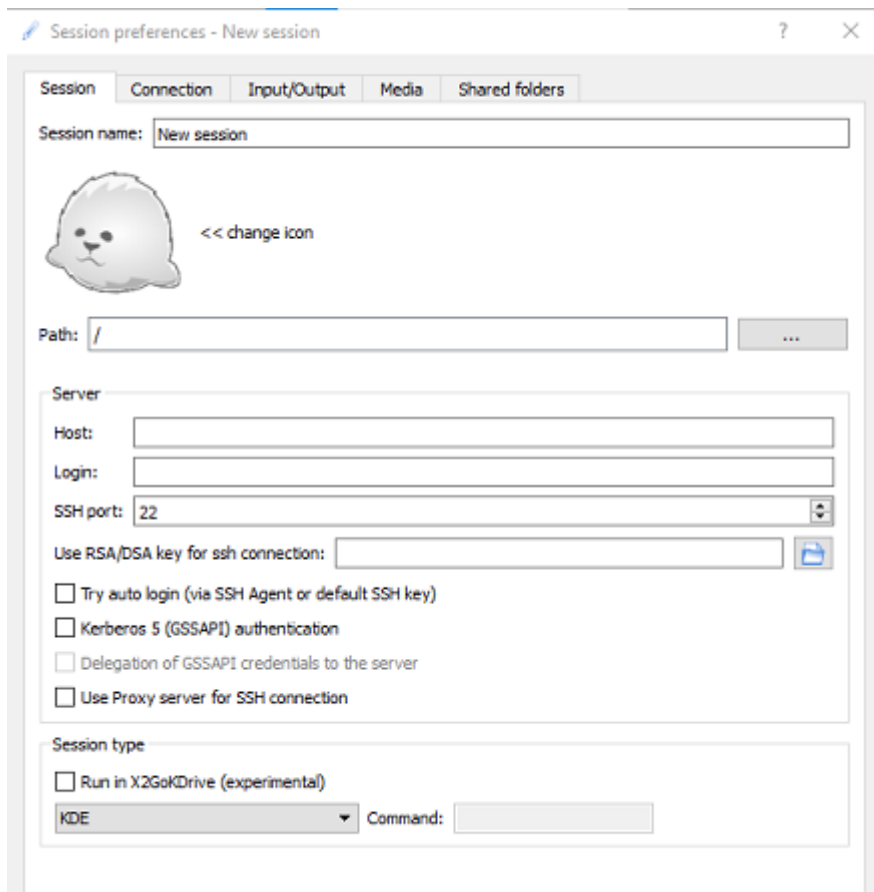
- OS: Ubuntu 20.04.6 LTS x86_64
- CPU: Intel Xeon Platinum 8260 (192) @ 3.900GHz
- Placa de vídeo apenas para suporte, não tem poder de processamento para outras atividades
- 04 sockets
- RAM: 1,5TB

Utilizaremos o programa X2go:

1. Clicar em New session

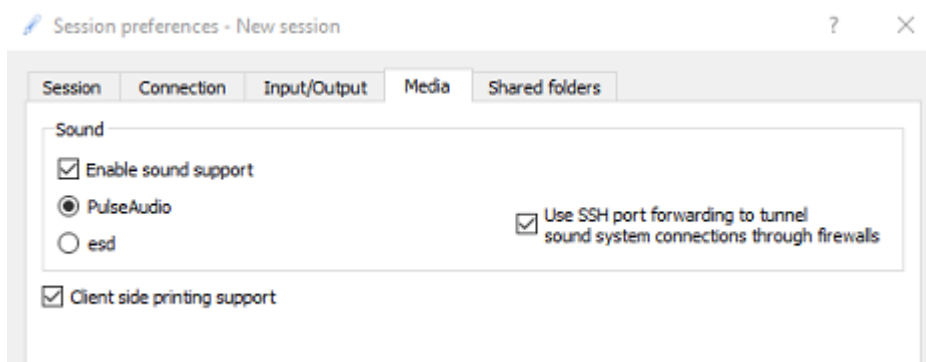


2. Preencher os campos:



Na aba “Session”

- Section name: Servidor ICC
- Host: 10.41.58.8
- Login:
- Em “Section type” selecionar: LXDE

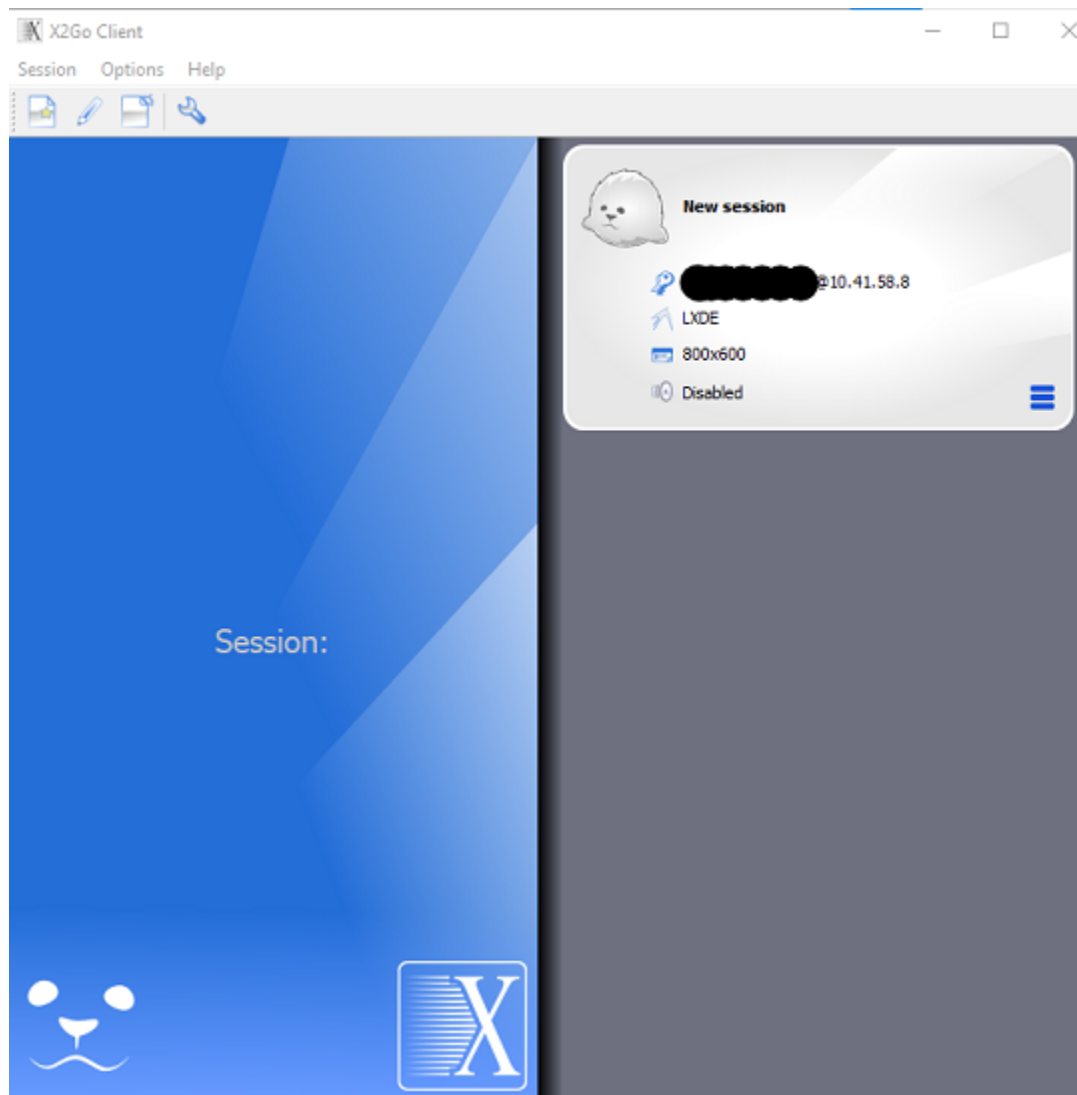


Na aba “Media”:

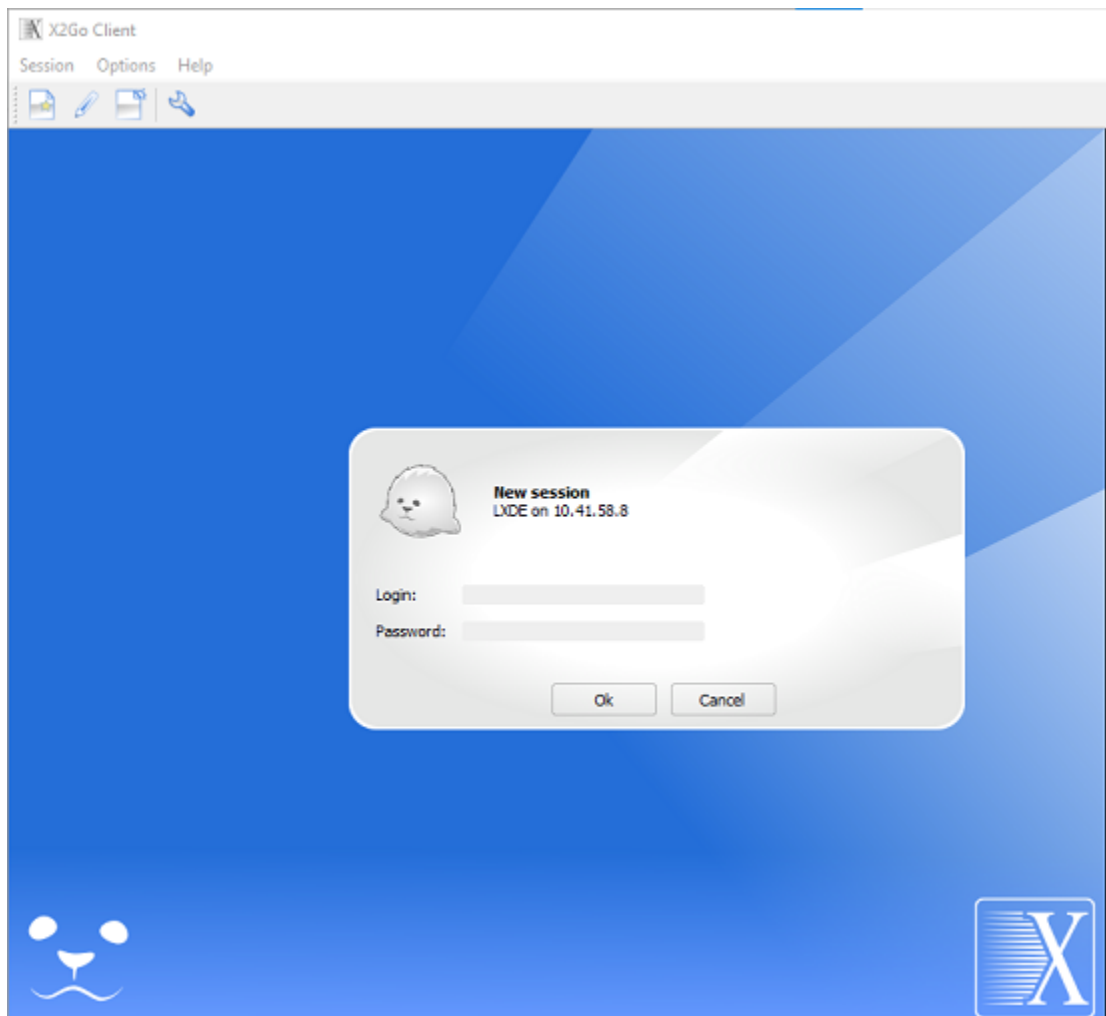
- Desmarcar “Enable sound support”
- Desmarcar “Client side printing support”

Clicar em OK para salvar as configurações.

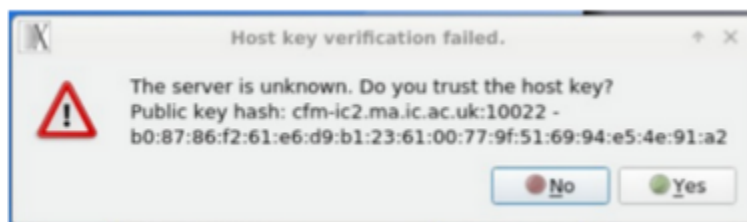
3. Acessar o servidor clicando na caixa lateral



Preencher o login e a senha (serão fornecidos em sala) e clique em OK.



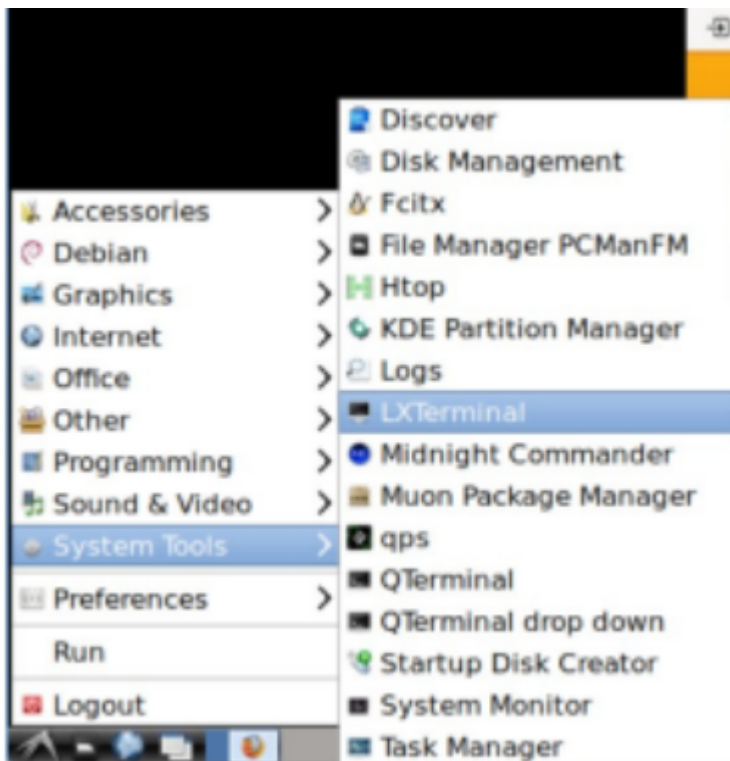
OBS: No primeiro acesso pode aparecer um alerta de “Host key verification failed”, basta clicar em **Yes**.



4. Se tudo deu certo uma nova janela como a da imagem abaixo vai aparecer:



5. Para acessar o Terminal clique no “menu inicial” (equivalente ao botão do Windows), vá em **System Tools** e em **LXTerminal**



2.2 Navegação Básica - Comando pwd

Informa a pasta atual.

Exemplo:

```
$ pwd
/home/aluno/curso-bioinfo
```

Atividade:

Execute `pwd` e anote qual o caminho completo do diretório onde você está.

2.3 Navegação Básica - Comando cd

Navegação entre diretórios:

- `cd ~` → Retorna ao diretório home do usuário.
- `cd {caminho relativo}` → Move para a pasta informada.
- `cd {caminho absoluto}` → Move para a pasta informada.
- `cd ..` → Move para o diretório pai.
- `cd .` → Diretório atual.

Exemplo:

```
$ cd data/
$ pwd
/home/aluno/curso-bioinfo/data
```

Atividade:

Navegue até a pasta `Desktop/segunda/` e depois volte à home (`cd ..`).

2.4 Manipulação de Diretórios e Arquivos - Comando ls

- `ls` → Lista arquivos e diretórios.
- `ls -la` → Lista arquivos e diretórios, incluindo ocultos e permissões.

Exemplo:

```
$ ls -la
drwxr-xr-x 2 aluno aluno 4096 jul 20 14:00 data
-rw-r--r-- 1 aluno aluno 234 jul 20 14:02 aula1.txt
```

Atividade:

Liste todos os arquivos, incluindo ocultos, do seu diretório atual.

2.5 Manipulação de Diretórios e Arquivos - Comando mkdir

Cria um diretório:

```
$ mkdir nome_do_diretorio
```

Exemplo:

```
$ mkdir resultados
```

Atividade:

Crie uma pasta `testes` dentro de `Desktop/segunda/`.

2.6 Manipulação de Diretórios e Arquivos - Comando `rmdir`

Remove um diretório (se estiver vazio):

```
$ rmdir nome_do_diretorio
```

Exemplo:

```
$ rmdir pasta_vazia
```

Atividade:

Remova a pasta temporária chamada *teste* com *rmdir*.

2.7 Abrindo um programa pela linha de comando - Comando `evince`

Como exemplo utilizaremos o programa “DocumentViewer” (leitor de pdf) que pode ser aberto utilizando o comando `evince`

Exemplo:

```
$ evince -help
Usage:
  evince [OPTION...] [FILE...] GNOME Document Viewer

Help Options:
  -h, --help                Show help options
  --help-all               Show all help options
  --help-gtk                Show GTK+ Options

Application Options:
  -p, --page-label=PAGE    The page label of the document to display.
  -i, --page-index=NUMBER The page number of the document to display.
  -n, --named-dest=DEST    Named destination to display.
  -f, --fullscreen         Run evince in fullscreen mode
  -s, --presentation       Run evince in presentation mode
  -w, --preview             Run evince as a previewer
  -l, --find=STRING        The word or phrase to find in the document
  --display=DISPLAY        X display to use
```

Dica: o terminal fica rodado o programa, então não recebe outros comandos até que o programa seja fechado, por exemplo.

Cada programa tem seu comando único.

Na dúvida usar comando `--help` ou comando `-h` mostra como funciona o comando

Exemplo:

```
$ evince arquivo.pdf
```

Atividade:

Execute *evince --help* ou *evince arquivo.pdf* para abrir essa apostila

2.8 Navegação Básica - Comando `man`

Acessa o manual do comando. A maioria dos comandos possui manual.

Exemplo:

```
$ man ls
```

Atividade:

Acesse o manual do comando `mkdir` ou outro comando usado anteriormente. Saia do manual e acesse o manual do comando `man`.

2.9 Navegação Básica - Comando `ssh`

Uso:

```
$ ssh {usuario}@{servidor}
```

Para suporte gráfico:

```
$ ssh -X {usuario}@{servidor}
```

Exemplo:

```
$ ssh -X aluno@bioinfo05
```

Atividade:

Conecte-se a uma máquina remota disponível na aula (ou simule o comando se for local).

2.10 Manipulação de Diretórios e Arquivos - Comando `touch`

Cria um novo arquivo:

```
$ touch nome_do_arquivo
```

Exemplo:

```
$ touch novo_script.sh
```

Atividade:

Crie um arquivo vazio chamado `README.txt`.

2.11 Manipulação de Diretórios e Arquivos - Comando `echo`

Escreve na saída padrão ou arquivos:

```
$ echo "mensagem" > arquivo.txt
```

- > substitui o conteúdo do arquivo.
- >> adiciona conteúdo ao já existente.

Exemplo:

```
$ echo "Olá bioinformática" > mensagem.txt
```

Atividade:

Use `echo` para criar um arquivo `mensagem.txt` com uma frase dentro na sua `home`.

2.12 Manipulação de Diretórios e Arquivos - Comando `cat`

Exibe o conteúdo de um arquivo:

```
$ cat arquivo.txt
```

Para concatenar arquivos podemos fazer assim:

```
$ cat arq1.txt arq2.txt > combinado.txt
```

Atenção: o `cat` pode não ser adequado para arquivos muito longos. Nesses casos usar: `head`, `tail`, `less` etc

Exemplo:

```
$ cat mensagem.txt
Olá bioinformática
```

Atividade:

Exiba o conteúdo do `mensagem.txt` criado anteriormente.

2.13 Manipulação de Diretórios e Arquivos - Comando `cp`

Copia arquivos:

```
$ cp origem destino
```

Exemplo:

```
$ cp aula1.txt copia_aula1.txt
```

Atividade:

Copie o arquivo `mensagem.txt` para dentro da pasta `Desktop/segunda`.

2.14 Manipulação de Diretórios e Arquivos - Comando `mv`

Mova arquivos e diretórios:

```
$ mv origem destino
```

Exemplo:

```
$ mv copia_aula1.txt backup_aula1.txt
```

Atividade:

Crie (com `echo` ou `touch` um novo arquivo `txt` na sua home. Mova `log.txt` para a pasta `Desktop/segunda`.

2.15 Manipulação de Diretórios e Arquivos - Comando `rm`

Remove arquivos:

```
$ rm arquivo.txt
```

- Para apagar diretórios recursivamente (CUIDADO!):

```
$ rm -r pasta/
```

- Para forçar a exclusão:

```
$ rm -rf pasta/
```

- Para remoção segura (para informações sensíveis):

```
$ shred -v -u arquivo.txt
```

Esse comando sobrescrever arquivos e discos de forma segura, tornando muito difícil ou impossível a recuperação dos dados, mesmo com ferramentas forenses. É comum o uso das opções:

- `-u`: Remove o arquivo após a sobrescrita.
- `-v`: Mostra o progresso da operação.

Exemplo:

```
$ rm backup_aula1.txt
```

Atividade:

Apague o arquivo `mensagem.txt` da sua home

2.16 Download de Arquivos - Comando `wget`

O comando `wget` é utilizado para realizar o download de arquivos diretamente via terminal, a partir de links HTTP, HTTPS ou FTP. Ele é muito útil para baixar genomas, dados de expressão e softwares em bioinformática.

Sintaxe básica:

```
wget https://url.com/arquivo
```

Faz o download do arquivo mantendo o nome original.

Você pode definir o nome do arquivo de saída com a opção `-O`:

```
$ wget -O nome_do_arquivo https://url.com/arquivo
```

Atenção: Ao renomear, preferencialmente mantenha a extensão original do arquivo (.gz, .fasta, .fastq, etc.), para garantir que outros programas reconheçam corretamente o tipo de arquivo.

Exemplo:

```
$ wget -O homegithub.html https://github.com/

--2025-07-29 13:50:40--  https://github.com/
Resolvendo github.com (github.com)... 20.201.28.151
Conectando-se a github.com (github.com)|20.201.28.151|:443... conectado.
A requisição HTTP foi enviada, aguardando resposta... 200 OK
Tamanho: não especificada [text/html]
Salvando em: 'homegithub.html'

homegithub.html [ <=>

2025-07-29 13:50:40 (10,1 MB/s) - 'homegithub.html' salvo [546852]
```

Atividade:

Baixe um arquivo público pequeno da internet, como um *.fasta* de um genoma exemplo. Por exemplo a EBI/ENA disponibiliza milhares de arquivos públicos, para acessar um exemplo:

Site: <https://www.ebi.ac.uk/ena/browser/view/PRJEB37886>

Link para o `wget`: <ftp://ftp.sra.ebi.ac.uk/vol1/fastq/ERR100/000/ERR10000000/ERR10000000.fastq.gz>

2.17 Manipulação de Diretórios e Arquivos - Comando `head` e `tail`

Visualiza as primeiras ou últimas linhas de um arquivo – útil para arquivos *.fastq*, *.fasta*, *.sam*, *.tsv*.

```
$ head arquivo.txt
$ tail arquivo.txt
$ head -n 20 arquivo.txt
$ tail -n 50 arquivo.txt
```

Exemplo:

```
$ head dados.tsv
```

Atividade:

Visualize as primeiras linhas do arquivo baixado anteriormente (um *html*, por exemplo).

2.18 Limpar a tela - `clear` ou `Ctrl+L`

Durante o uso do terminal, é comum precisar limpar a tela para facilitar a visualização ou interromper um comando em execução.

O comando `clear` limpa o conteúdo da tela do terminal, sem encerrar a sessão atual. Exemplo:

```
clear
```

Também é possível usar o atalho `Ctrl + L` para obter o mesmo efeito em muitas distribuições Linux.

2.19 Manipulação de Diretórios e Arquivos - Comando `less`

Visualizador de arquivos grandes com navegação:

```
$ less arquivo.txt
```

Teclas úteis: `Q` para sair, `/` para buscar, `G` para ir ao fim.

Exemplo:

```
$ less dados_grandes.txt
```

Atividade:

Use `less` para navegar em um `.fastq` ou arquivo de texto grande.

2.20 Transferência de Arquivos entre computadores - Comando `scp`

Transferência entre computadores:

- Enviar do local para o remoto:

```
$ scp arquivo.txt usuario@servidor:/caminho/destino
```

- Trazer do remoto para o local:

```
$ scp usuario@servidor:/caminho/arquivo.txt destino_local
```

Para grandes volumes de dados, considere `rsync`.

Exemplo:

```
$ scp arquivo.txt aluno@bioinfo05:/home/aluno/
```

Atividade:

Simule ou pratique a transferência de um arquivo entre duas máquinas (se possível).

2.21 Monitorando processos - Comando `top`

O comando `top` permite monitorar em tempo real os processos em execução no sistema. Ele exibe informações como uso de CPU, memória, tempo de execução, entre outros — sendo útil para diagnosticar gargalos e acompanhar o desempenho da máquina durante a execução de tarefas pesadas, como análise de dados ou montagem genômica.

Uso básico:

```
$ top
```

```
top - 12:42:24 up 40 min, 1 user, load average: 0,51, 0,35, 0,41
Tarefas: 298 total, 1 em exec., 297 dormindo, 0 parado, 0 zumbi
%CPU(s): 6,2 us, 1,4 sy, 0,2 ni, 91,5 id, 0,6 wa, 0,0 hi, 0,0 si, 0,0 st
MB mem : 15768,8 total, 1230,8 livre, 4888,6 usados, 9649,4 buff/cache
MB swap: 2048,0 total, 2048,0 livre, 0,0 usados, 10371,2 mem dispon.
```

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPO+	COMANDO
1792	helena	20	0	6140728	292284	143168	S	20,2	1,8	1:55.94	gnome-shell
46834	helena	20	0	636860	47228	36200	S	5,3	0,3	0:00.16	nautilus
46838	helena	20	0	912664	61288	46272	S	5,3	0,4	0:00.16	gnome-calendar
46842	helena	20	0	394344	41432	26144	S	5,3	0,3	0:00.16	seahorse
46840	helena	20	0	2774084	45252	33924	S	3,0	0,3	0:00.09	org.gnome.Chara
46832	helena	20	0	344544	24576	17536	S	2,0	0,2	0:00.06	gnome-control-c
46836	helena	20	0	417684	24324	17412	S	2,0	0,2	0:00.06	gnome-calculato
5000	helena	30	10	724812	226396	116164	S	1,7	1,4	0:10.34	update-manager
5551	helena	20	0	4091504	1,2g	117372	S	1,7	8,0	4:06.55	Isolated Web Co
46876	root	20	0	14700	6656	5888	S	1,7	0,0	0:00.05	systemd-timedat
5133	helena	20	0	11,7g	611548	235236	S	1,3	3,8	3:56.27	firefox
1656	helena	20	0	10112	5632	4096	S	1,0	0,0	0:00.61	dbus-daemon
1953	helena	20	0	316176	12288	7296	S	0,7	0,1	0:03.38	ibus-daemon
2049	helena	20	0	349856	31712	20340	S	0,7	0,2	0:01.43	ibus-extension-
1	root	20	0	315812	13440	8320	S	0,3	0,1	0:03.91	systemd
70	root	20	0	0	0	0	I	0,3	0,0	0:01.02	kworker/u16:2-i915
1641	helena	20	0	241280	7324	6656	S	0,3	0,0	0:00.04	gnome-keyring-d
1789	helena	20	0	8564	4352	3968	S	0,3	0,0	0:00.05	dbus-daemon
1897	helena	20	0	236780	6656	6144	S	0,3	0,0	0:00.01	gvfs-mtp-volume
2219	helena	39	19	719996	38740	25880	S	0,3	0,2	0:01.97	tracker-miner-f
2319	helena	20	0	3075556	74264	54660	S	0,3	0,5	0:01.72	gjs
3103	helena	20	0	942948	318084	77056	S	0,3	2,0	0:15.14	rsession
5461	helena	20	0	2473592	127356	87392	S	0,3	0,8	0:01.57	Privileged Cont
5569	helena	20	0	2943400	442756	98656	S	0,3	2,7	0:10.81	Isolated Web Co
25550	systemd+	20	0	14836	6784	6016	S	0,3	0,0	0:02.86	systemd-oomd
46499	root	20	0	0	0	0	I	0,3	0,0	0:00.06	kworker/5:2-i915-unorder
46595	helena	20	0	2452404	83456	64512	S	0,3	0,5	0:00.16	Isolated Servic
46722	helena	20	0	559216	57420	44204	S	0,3	0,4	0:00.31	gnome-terminal-
46773	helena	20	0	2423804	70016	55296	S	0,3	0,4	0:00.05	Web Content
46811	helena	20	0	13484	4352	3456	R	0,3	0,0	0:00.05	top
2	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-rcu_g
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-rcu_p
6	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-slub_
7	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-netns
9	root	0	-20	0	0	0	I	0,0	0,0	0:00.14	kworker/0:0H-kblockd
12	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-mm_pe

Exemplo

A interface interativa exibida pelo top inclui:

- PID: identificador do processo.
- USER: usuário que iniciou o processo.
- %CPU: percentual de uso da CPU.
- %MEM: percentual de uso da memória RAM.
- TIME+: tempo total de CPU utilizado.
- COMMAND: nome do comando executado.

Enquanto o top está aberto, você pode usar algumas teclas para interagir:

- q: sair do top.
- P: ordenar por uso de CPU (padrão).
- M: ordenar por uso de memória.
- k: matar (encerrar) um processo (será solicitado o PID).
- h: ajuda.

Atividade:

Execute os seguintes comandos no terminal:

```
$ top
```

Observe se o processo aparece nas primeiras posições, e como ele impacta o uso da CPU e memória do sistema. Para ver os processos de um único usuário:

```
$ top -u $USER
```

Ou para atualizar mais lentamente (útil para evitar flutuações):

```
$ top -d 5
```

2.22 Criação de Ambiente de Trabalho – Comandos `screen` ou `tmux`

Essas ferramentas permitem gerenciar múltiplas sessões no terminal, úteis para executar tarefas demoradas (como downloads, pipelines de bioinformática ou montagem de genomas) sem interrupções mesmo que a conexão com o servidor caia.

- Iniciar nova sessão:

```
$ screen -S nome_da_sessao
```

- Listar sessões:

```
$ screen -ls
```

- Reconectar:

```
$ screen -r nome_da_screen
```

- Navegação entre sessões:

- Ctrl + A + P → Sessão anterior
- Ctrl + A + N → Próxima
- Ctrl + A + C → Nova
- Ctrl + A + D → Desconectar
- Ctrl + A + K → Encerrar

Encerrar a sessão apenas com `exit` ou Ctrl + A + K. Desconectar (Ctrl + A + D) mantém a sessão rodando em segundo plano.

Exemplo:

```
$ screen
# (em nova sessão)
$ top
```

2.22.1 Alternativa: Comando `tmux`

O `tmux` é uma alternativa mais moderna ao `screen`, com recursos avançados de divisão de janelas e personalização.

Iniciar nova sessão:

```
$ tmux new -s nome_da_sessao
```

- Criar nova janela dentro da sessão: Ctrl + B, C

- Alternar entre janelas:
 - Ctrl + B, N # próxima
 - Ctrl + B, P # anterior
- Dividir a janela atual:
 - Ctrl + B, " # divide horizontalmente
 - Ctrl + B, % # divide verticalmente
- Navegar entre painéis:
 - Ctrl + B, seta direcional
- Desconectar da sessão: Ctrl + B, D

Listar sessões e reconectar:

```
$ tmux ls
$ btmux attach -t nome_da_sessao
```

Como no screen, os processos continuam rodando mesmo após a desconexão.

Atividade: Abra uma nova screen ou tmux, rode `sleep 60`, desconecte (Ctrl+A+D ou Ctrl+B,D), e reconecte à sessão. Observe se o processo continua em execução.

2.23 Compactação de Arquivos - Comando gzip e gunzip

Compressão e descompressão de arquivos:

```
$ gzip arquivo.fastq
$ gunzip arquivo.fastq.gz
```

Exemplo:

```
$ gzip aula1.txt
```

Atividade:

Compacte um dos arquivos de texto criados em aula.

2.24 Compactação de Arquivos - Comando tar

Empacota e comprime diretórios:

```
$ tar -czvf pacote.tar.gz pasta/
$ tar -xzvf pacote.tar.gz
```

Exemplo:

```
$ tar -czvf backup.tar.gz scripts/
```

Atividade:

Empacote toda a pasta `scripts/` em um `.tar.gz`.

2.25 Monitoramento de tempo de execução - Comando time

O comando `time` é utilizado para medir o tempo que um processo leva para ser executado. Ele pode ser particularmente útil em bioinformática para avaliar o desempenho de programas e scripts que processam grandes volumes de dados.

Exemplo:

```
$ time ls -l
```

Esse comando listará os arquivos do diretório com detalhes (`ls -l`) e, ao final, exibirá quanto tempo a operação levou para ser executada.

A saída típica do `time` normalmente terá:

- **real**: tempo total decorrido (tempo de relógio de parede).
- **user**: tempo de CPU gasto em modo usuário (tempo gasto executando código do programa).
- **sys**: tempo de CPU gasto em modo sistema (tempo gasto com chamadas do sistema).

Exemplo:

```
real    0m0.002s
user    0m0.000s
sys     0m0.002s
```

Também é possível redirecionando o tempo para um arquivo a parte:

```
{ time comando; } 2> tempo.txt
```

Atividade:

Compare o tempo necessário para compactar dois tipos diferentes de arquivos com `gzip`. Passo a passo: 1. Navegue até a pasta onde estão os arquivos:

```
$ cd Desktop/segunda/
```

2. Verifique os tamanhos dos arquivos antes da compressão:

```
$ ls -la
```

3. Meça o tempo para compactar cada arquivo:

```
$ time gzip -k arquivo.html
$ time gzip -k arquivo.fasta
```

4. (Opcional) Verifique o tamanho dos arquivos `.gz` gerados:

```
$ ls -la *.gz
```

2.26 Processamento de Dados em Tabelas - Comandos `cut`, `sort`, `uniq`, `wc`, `grep`

Comandos fundamentais para manipulação de arquivos tabulares e listas:

```
$ cut -f1,3 dados.tsv
$ sort arquivo.txt
$ uniq -c
$ wc -l arquivo.txt
$ grep "geneX" arquivo.gff
```

Combine com `|` para criar minipipelines de análise.

Exemplo:

```
$ cut -f1,2 dados.tsv
```

Atividade:

Crie um arquivo com 3 colunas fictícias e use `cut` para extrair só a primeira.

2.27 Permissões e Ambiente - Comando `chmod`

Concede permissão de execução a scripts:

```
$ chmod +x meu_script.sh
```

Exemplo:

```
$ chmod +x novo_script.sh
```

Atividade:

Crie um arquivo de script (.sh) e torne-o executável.

2.28 Permissões e Ambiente - Comando which

Mostra o caminho de um comando instalado:

```
$ which blastn
```

Exemplo:

```
$ which blastn
/usr/bin/blastn
```

Atividade:

Verifique onde está instalado *wget* ou outro comando já usado.

2.29 Permissões e Ambiente - Comando history

Exibe histórico de comandos do terminal:

```
$ history
```

Exemplo:

```
$ history | tail
105  ls
106  cd scripts
```

Atividade:

Revise os últimos comandos que você usou hoje e salve os mais importantes em um arquivo chamado *comandos.txt*.

2.30 Encontrar Arquivos e/ou Diretórios - Comando find

O comando find permite localizar arquivos e diretórios no sistema, com base em critérios como nome, tipo ou data de modificação.

Estrutura geral:

```
$ find [options] [path...] [expression]
```

Exemplos práticos:

Procurando um arquivo chamado “document.pdf” na pasta de Downloads:

```
$ find /home/login/Downloads/ -type f -name document.pdf
```

Repare que:

- - **type f** indica que estamos procurando um arquivo (= file)
- - **name** é porque estamos procurando baseado no nome do arquivo (é sensível a maiúscula/minúscula)
- - **iname** procura no nome sem considerar maiúsculas e minúsculas

```
$ find /home/login/Downloads/ -type f -iname document.pdf
```

Podemos usar isso para achar todos os genomas em FASTA (.fna) numa pasta por exemplo:

```
$ find /home/login/genomas/ -type f -iname "*.fna"
```

Caso eu queira salvar o resultado num arquivo posso fazer assim:

```
$ find /home/login/genomas/ -type f -iname "*.fna" > lista_genomas.txt
```

Exemplos práticos:

1. Procurar um arquivo chamado document.pdf na pasta de Downloads:

```
$ find /home/login/Downloads/ -type f -name document.pdf
```

- `-type f` procura apenas arquivos (f = file).
- `-name` busca pelo nome exato (diferencia maiúsculas e minúsculas).

2. Ignorar maiúsculas/minúsculas:

```
$ find /home/login/Downloads/ -type f -iname document.pdf
```

- `-iname` busca insensível a maiúsculas/minúsculas.

3. Buscar todos os arquivos .fna em uma pasta:

```
$ find /home/login/genomas/ -type f -iname "*.fna"
```

4. Salvar os resultados em um arquivo:

```
$ find /home/login/genomas/ -type f -iname "*.fna" >> lista_genomas.txt
```

Importante: diferença entre > e >>

- `>` Sobrescreve o conteúdo do arquivo. Se o arquivo já existir, será apagado e substituído pelo novo conteúdo.
- `>>` Adiciona ao final do arquivo. Preserva o conteúdo anterior e insere novas linhas.

Use > com cuidado para não apagar arquivos acidentalmente!

2.31 Outros Exemplos úteis de comandos em Bioinformática

- Converter um arquivo FASTQ para FASTA

```
$ sed -n '1-4s/^@/>/p;2~4p' file.fq > file.fa
```

- Determine o número de genes anotados em um arquivo GFF3

```
$ grep -c '$\tgene\t' yourannotations.gff3
```

- Renomeie todos os arquivos .txt para .bak (Exemplo: faça backup dos arquivos .txt antes de executar qualquer comando que os altere.)

```
$ find . -name "*.txt" | sed "s/\.txt$//" | xargs -i echo mv {}.txt {}.bak | sh
```

- Calcule o comprimento médio das leituras em um arquivo fastq

```
$ awk 'NR%4==2{sum+=length($0)}END{print sum/(NR/4)}' input.fastq
```

Execute o FastQC em paralelo, 12 tarefas por vez:

```
$ find *.fq | parallel -j 12 "fastqc {} --outdir ."
```

Fonte e mais comandos em: <https://github.com/stephenturner/oneliners/blob/master/README.md>

2.32 Atividade extra

Atividade:

1. Crie uma pasta *genomas/*
2. Baixe pelo terminal alguns genomas *.fna*, *.txt* e *.fasta* e coloque nessa pasta.
3. Liste todos os arquivos *.fna*
4. Salve o resultado em um arquivo *lista_genomas.txt*
5. Adicione novas buscas ao mesmo arquivo, sem sobrescrevê-lo
6. Visualize o conteúdo final do arquivo no terminal
7. Faça a versão comprimida *.gz* do seu arquivo
8. Liste os arquivos na *genomas* no terminal com suas propriedades e tamanhos